

Smart Expense And Budget Analyzer

PROJECT REPORT

Submitted by: Prishita Singh Thakur

Reg No- 26BAI10484

Academic Year- 2026-27

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

Managing personal finances is a challenge many individuals face on a daily basis. Without a simple way to record and review spending, it becomes difficult to know where money is going or whether one is staying within a planned budget. The “Smart Expense And Budget Analyzer” is a menu-driven, console-based Python application built to address this problem in a lightweight and accessible way.

This project was built as part of the VITyarthi “Build Your Own Project” flipped-course evaluation. It applies core subject concepts modular programming, object-oriented design, dictionaries, loops, and exception handling to a real, relatable problem: tracking daily expenses and comparing them against a monthly budget.

The application is organised into five Python files (main.py, expense.py, budget.py, analyzer.py, and utils.py), each responsible for a distinct part of the system, which keeps the codebase clean, testable, and easy to extend.

2. Problem Statement

Most people do not maintain a structured record of their day-to-day expenses. As a result, they are often unaware of how much they have spent in a given month, which category consumes the most money, or whether they have exceeded their intended budget until it is too late. Spreadsheet tools exist, but they are often heavier than needed for a quick, personal, single-session check of spending.

This project aims to solve that problem by providing a simple, no-installation, no-internet, command-line tool that lets a user log expenses as they happen, set a monthly budget, and instantly see whether they are within budget along with a category-wise breakdown of where the money went.

3. Functional Requirements

In line with the project guidelines, the application is built around three major functional modules, each with a clear input/output structure:

Module	Input	Processing	Output
Expense Management	Amount, category (from a fixed list), description	Validates the amount and category; wraps the data in an Expense object	Confirmation message; expense stored in memory and listable on demand
Budget Management	Monthly budget amount	Compares total recorded expenses against the set budget	Remaining balance, or the amount by which the budget has been exceeded
Expense Analysis	The list of recorded expenses (no separate input)	Aggregates total spending and groups amounts by category	Total spending and a percentage-wise, category-wise breakdown

Logical Workflow

- The user is presented with a Main Menu offering six options.
- Selecting Add Expense collects and validates an amount, category, and description, then stores it.
- Selecting View Expenses lists every expense recorded so far in the current session.
- Selecting Set Budget records the user's intended monthly spending limit.
- Selecting Check Budget compares total spending against that limit and reports the outcome.
- Selecting Expense Analysis reports total spending and its category-wise percentage split.
- Selecting Exit ends the program gracefully with a closing message.

4. Non-Functional Requirements

Beyond core functionality, the project was designed with the following non-functional qualities in mind:

- **Usability:** A clearly labelled, numbered menu and prompts (with the ₹ symbol) make the tool approachable for a non-technical user.
- **Reliability:** Every numeric input (amount, budget, category choice) is wrapped in a validation loop, so the program does not crash on bad input — it simply re-prompts.
- **Maintainability:** The modular file structure (one responsibility per file) means a feature can be modified or extended without touching unrelated code.
- **Performance:** All operations run on in-memory Python lists and dictionaries, so menu actions respond instantly even with a large number of expense entries.
- **Error-Handling Strategy:** try/except ValueError blocks around every numeric input, combined with explicit range checks (amount > 0, budget > 0, category within list bounds), prevent invalid data from entering the system.
- **Resource Efficiency:** The application uses only Python's standard library — no external dependencies — keeping it lightweight and easy to run anywhere Python 3 is installed.

5. System Architecture

The application follows a simple layered architecture. The Presentation Layer (`main.py`) handles the menu and program flow. It delegates work to the Business Logic Layer, made up of `expense.py`, `budget.py`, and `analyzer.py`, which contain the actual rules for creating expenses, managing the budget, and analysing spending. A shared Utility Layer (`utils.py`) provides reusable, validated input helpers used across the application. Finally, an in-memory Data Layer — a plain Python list of expense dictionaries — holds all data for the duration of a single run.

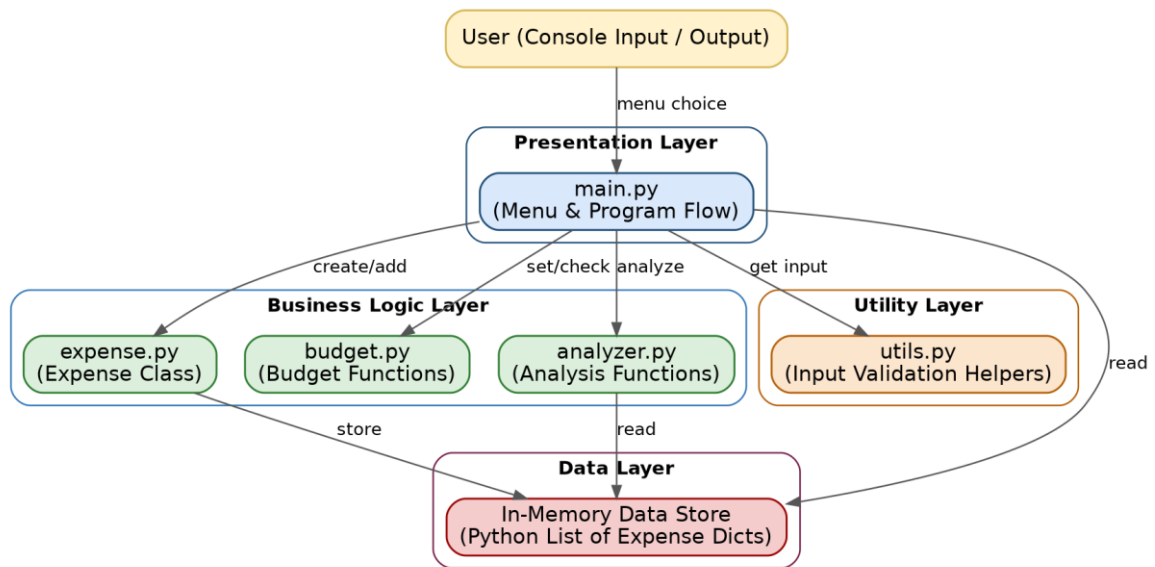
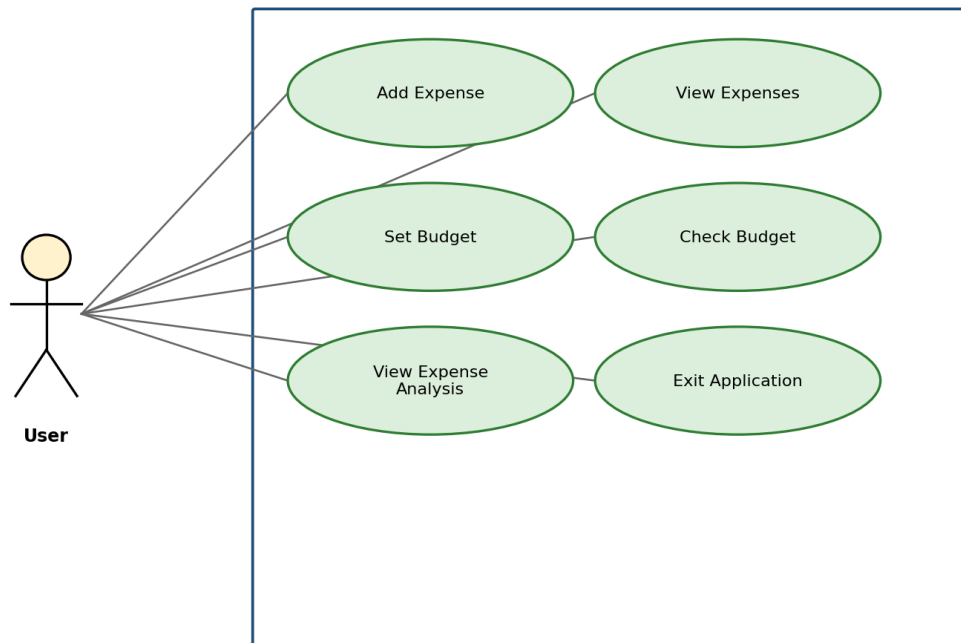


Figure 5.1: Layered system architecture of the Smart Expense Tracker & Budget Analyzer

6. Design Diagrams

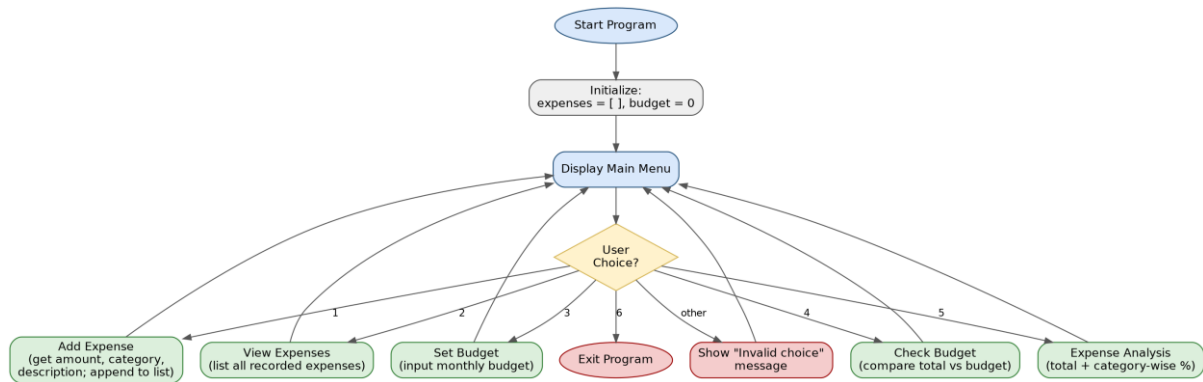
6.1 Use Case Diagram

The diagram below shows the single actor (User) and the six use cases the system supports, all contained within the application's system boundary.



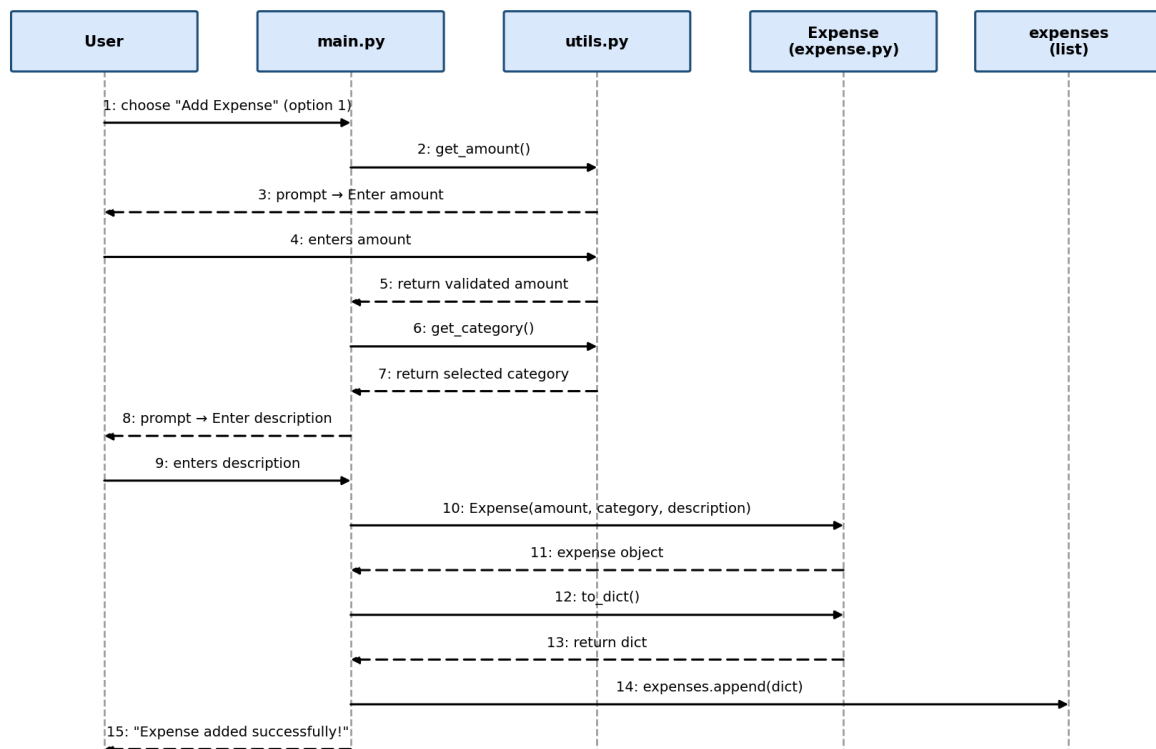
6.2 Workflow / Process Flow Diagram

This flowchart illustrates the overall program flow, from initialization through the main menu loop, to each of the six possible user actions, and back to the menu until the user exits.



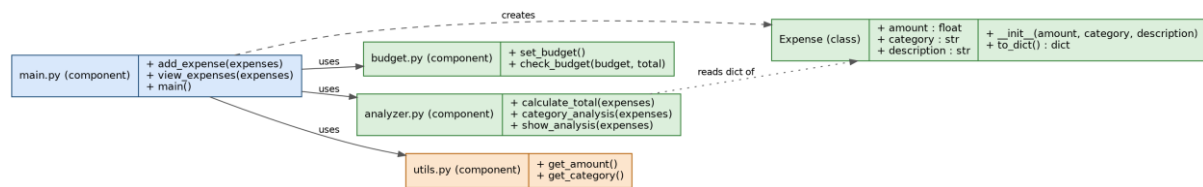
6.3 Sequence Diagram — Add Expense

The sequence diagram below traces the interaction that occurs when a user selects “Add Expense” from the menu, showing the message flow between the user, main.py, utils.py, the Expense class, and the in-memory expenses list.



6.4 Class / Component Diagram

The diagram below shows the Expense class along with its attributes and methods, and the dependency relationships between the four module-level components.



6.5 ER Diagram

Not applicable to this version of the project. The application does not use a database or a persistent file store all expense and budget data is held in memory (Python lists and variables) for the duration of a single program run and is intentionally not persisted, so no entity-relationship model is required. This is listed as a planned enhancement in Section 13 (Future Enhancements).

7. Design Decisions & Rationale

- **Modular file structure:** Splitting the code into main.py, expense.py, budget.py, analyzer.py, and utils.py was chosen over a single script so each concern (flow control, data modelling, budgeting, analysis, input handling) can be developed, tested, and understood independently.
- **Object-oriented Expense model:** Expense is implemented as a class rather than a plain dictionary so that expense data and its behaviour (to_dict()) are bundled together, making the code easier to extend (e.g., adding a date field or a display method later).
- **Fixed category list:** Categories (Food, Transport, Shopping, Education, Entertainment, Other) are presented as a numbered list rather than free text, to avoid inconsistent spelling/casing that would otherwise break the category-wise analysis.
- **In-memory storage:** For this version, expenses are kept in a simple Python list rather than a file or database, keeping the project's scope focused on the core logic requested by the assignment; persistence is identified as a future enhancement.
- **No external libraries:** The project intentionally relies only on Python's standard library, so it runs on any machine with Python 3 installed, with no setup or installation step.

8. Implementation Details

The project is implemented in Python 3 using five files. `main.py` contains `add_expense()`, `view_expenses()`, and `main()`, which together drive the menu loop and dispatch to the other modules based on the user's numbered choice. `expense.py` defines the `Expense` class, whose `to_dict()` method converts an expense object into a plain dictionary so it can be stored in, and iterated over, a simple list. `budget.py` exposes `set_budget()` and `check_budget()`, which respectively collect the monthly limit and compare it against total spending. `analyzer.py` provides `calculate_total()`, `category_analysis()`, and `show_analysis()`, which together compute the overall and per-category spending figures shown to the user. `utils.py` centralises the two input-validation helpers, `get_amount()` and `get_category()`, which are reused wherever numeric or categorical input is required, avoiding duplicated validation logic.

Exception handling is applied consistently: every place that converts user input to a number (`float()` or `int()`) is wrapped in a `try/except ValueError` block, and business-rule checks (`amount > 0`, `budget > 0`, a category index within range) are enforced with explicit `if` conditions before the value is accepted. The full annotated source code for all five files is reproduced below.

main.py

```
from expense import Expense
from budget import set_budget, check_budget
from analyzer import calculate_total, show_analysis
from utils import get_amount, get_category

def add_expense(expenses):
    print("\n===== ADD EXPENSE =====")

    amount = get_amount()
    category = get_category()

    description = input("Enter description: ")

    expense = Expense(amount, category, description)

    expenses.append(expense.to_dict())

    print("\n✓ Expense added successfully!")

def view_expenses(expenses):
    print("\n===== ALL EXPENSES =====")

    if not expenses:
        print("No expenses found.")
        return

    for i, expense in enumerate(expenses, start=1):
        print(
            f"{i}. ₹{expense['amount']:.2f} | "
            f"{expense['category']} | "
            f"{expense['description']}"
        )
```

```

    )

def main():

    # Fresh data every time the program starts
    expenses = []
    budget = 0

    print("\n┌──────────────────────────────────────────┐")
    print("│          SMART EXPENSE TRACKER          │")
    print("│          & BUDGET ANALYZER             │")
    print("└──────────────────────────────────────────┘")

    print("\n"Track it. Analyse it. Manage it.")

    while True:

        print("\n=====")
        print("          MAIN MENU")
        print("=====")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Set Budget")
        print("4. Check Budget")
        print("5. Expense Analysis")
        print("6. Exit")

        choice = input("\nEnter your choice: ")

        if choice == "1":
            add_expense(expenses)

        elif choice == "2":
            view_expenses(expenses)

        elif choice == "3":
            budget = set_budget()
            print("✓ Budget set successfully!")

        elif choice == "4":
            if budget == 0:
                print("\nPlease set your budget first.")
            else:
                total = calculate_total(expenses)
                check_budget(budget, total)

        elif choice == "5":
            show_analysis(expenses)

        elif choice == "6":
            print("\nThank you for using Smart Expense Tracker!")
            break

        else:
            print("\nInvalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

expense.py

```
class Expense:
    def __init__(self, amount, category, description):
        self.amount = amount
        self.category = category
        self.description = description

    def to_dict(self):
        return {
            "amount": self.amount,
            "category": self.category,
            "description": self.description
        }
```

budget.py

```
def set_budget():
    while True:
        try:
            budget = float(input("Enter your monthly budget: ₹"))

            if budget <= 0:
                print("Budget must be greater than zero.")
            else:
                return budget

        except ValueError:
            print("Please enter a valid number.")

def check_budget(budget, total_expense):
    remaining = budget - total_expense

    print("\n===== BUDGET STATUS =====")
    print(f"Budget: ₹{budget:.2f}")
    print(f"Spent: ₹{total_expense:.2f}")

    if remaining >= 0:
        print(f"Remaining: ₹{remaining:.2f}")
    else:
        print(f"Over budget by: ₹{abs(remaining):.2f}")
```

analyzer.py

```
def calculate_total(expenses):
    total = 0

    for expense in expenses:
        total += expense["amount"]

    return total

def category_analysis(expenses):
    categories = {}

    for expense in expenses:
        category = expense["category"]
        amount = expense["amount"]

        if category in categories:
```

```

        categories[category] += amount
    else:
        categories[category] = amount

    return categories

def show_analysis(expenses):
    if not expenses:
        print("\nNo expenses available for analysis.")
        return

    total = calculate_total(expenses)
    categories = category_analysis(expenses)

    print("\n==== EXPENSE ANALYSIS =====")
    print(f"Total Spending: ₹{total:.2f}")

    print("\nSpending by Category:")

    for category, amount in categories.items():
        percentage = (amount / total) * 100
        print(f"{category}: ₹{amount:.2f} ({percentage:.1f}%)")

```

utils.py

```

def get_amount():
    while True:
        try:
            amount = float(input("Enter amount: ₹"))

            if amount <= 0:
                print("Amount must be greater than zero.")
            else:
                return amount

        except ValueError:
            print("Please enter a valid amount.")

def get_category():
    categories = [
        "Food",
        "Transport",
        "Shopping",
        "Education",
        "Entertainment",
        "Other"
    ]

    print("\nCategories:")

    for i, category in enumerate(categories, start=1):
        print(f"{i}. {category}")

    while True:
        try:
            choice = int(input("Choose category: "))

            if 1 <= choice <= len(categories):
                return categories[choice - 1]

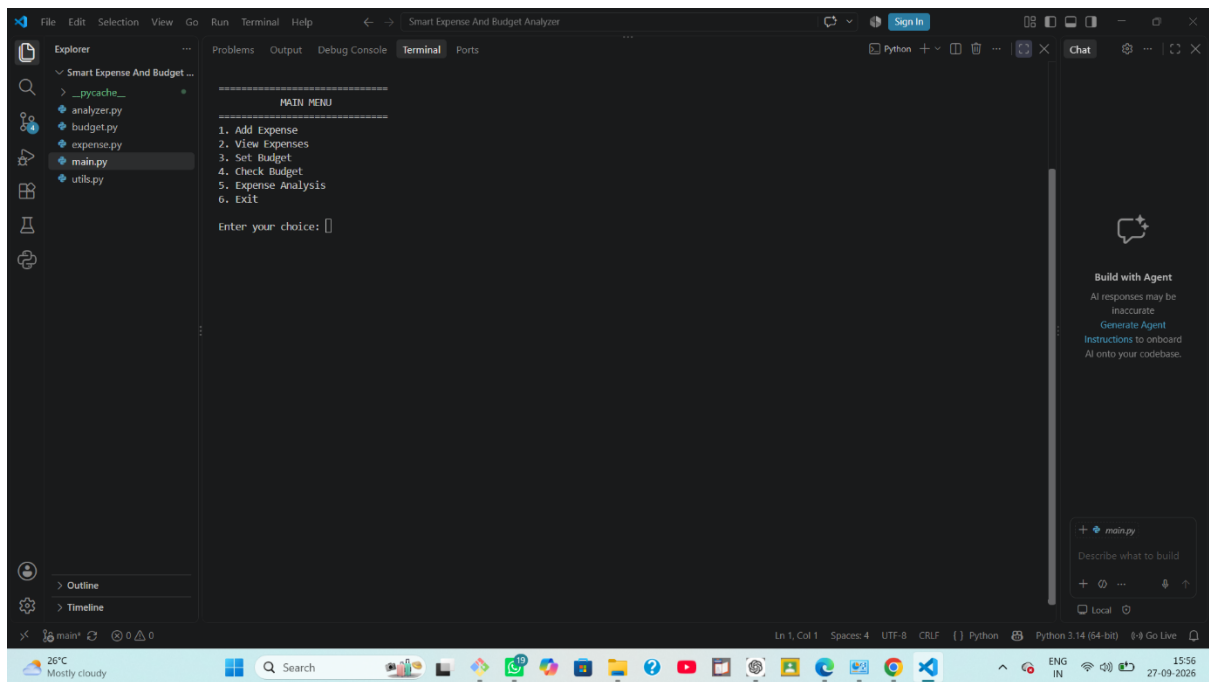
```

```
        else:
            print("Please choose a valid option.")

    except ValueError:
        print("Please enter a number.")
```

9. Screenshots / Results

Below is an illustrative sample of the console output produced when running the application



The screenshot shows a Windows terminal window titled "Smart Expense And Budget Analyzer". The terminal displays the following output:

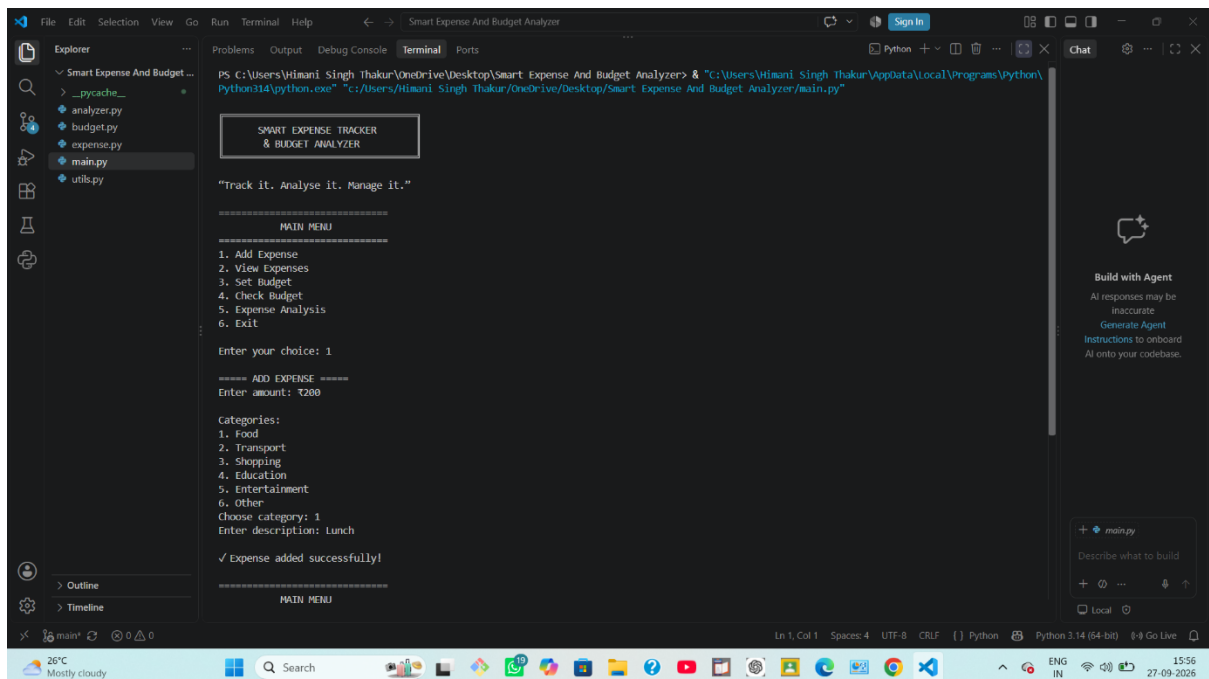
```
=====
MAIN MENU
=====
1. Add Expense
2. View Expenses
3. Set Budget
4. Check Budget
5. Expense Analysis
6. Exit

Enter your choice: 
```

The Explorer pane on the left shows the project structure:

- Smart Expense And Budget ...
 - __pycache__
 - analyzer.py
 - budget.py
 - expense.py
 - main.py
 - utils.py

The status bar at the bottom indicates "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Python 3.14 (64-bit)".



The screenshot shows the same terminal window after running the application. The output is as follows:

```
PS C:\Users\Himani Singh Thakur\OneDrive\Desktop\Smart Expense And Budget Analyzer> & "C:\Users\Himani Singh Thakur\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\Himani Singh Thakur\OneDrive\Desktop\Smart Expense And Budget Analyzer\main.py"

SMART EXPENSE TRACKER
& BUDGET ANALYZER

"Track it. Analyse it. Manage it."

=====
MAIN MENU
=====
1. Add Expense
2. View Expenses
3. Set Budget
4. Check Budget
5. Expense Analysis
6. Exit

Enter your choice: 1

===== ADD EXPENSE =====
Enter amount: ₹200

Categories:
1. Food
2. Transport
3. Shopping
4. Education
5. Entertainment
6. Other
Choose category: 1
Enter description: Lunch

✓ Expense added successfully!

=====
MAIN MENU
=====
```

The Explorer pane on the left remains the same. The status bar at the bottom indicates "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Python 3.14 (64-bit)".

```
1 from expense import Expense
2 from budget import set_budget, check_budget
3 from analyzer import calculate_total, show_analysis
4 from utils import get_amount, get_category
5
6
7 def add_expense(expenses):
8     print("\n==== ADD EXPENSE =====")
9
10    amount = get_amount()
11    category = get_category()
12
13    description = input("Enter description: ")
14
```

```
=====
MAIN MENU
=====
1. Add Expense
2. View Expenses
3. Set Budget
4. Check Budget
5. Expense Analysis
6. Exit

Enter your choice: 5

===== EXPENSE ANALYSIS =====
Total Spending: ₹380.00

Spending by Category:
Food: ₹280.00 (66.7%)
Transport: ₹100.00 (33.3%)
```

```
1 from expense import Expense
2 from budget import set_budget, check_budget
3 from analyzer import calculate_total, show_analysis
4 from utils import get_amount, get_category
5
6
7 def add_expense(expenses):
8     print("\n==== ADD EXPENSE =====")
9
10    amount = get_amount()
11    category = get_category()
12
13    description = input("Enter description: ")
14
```

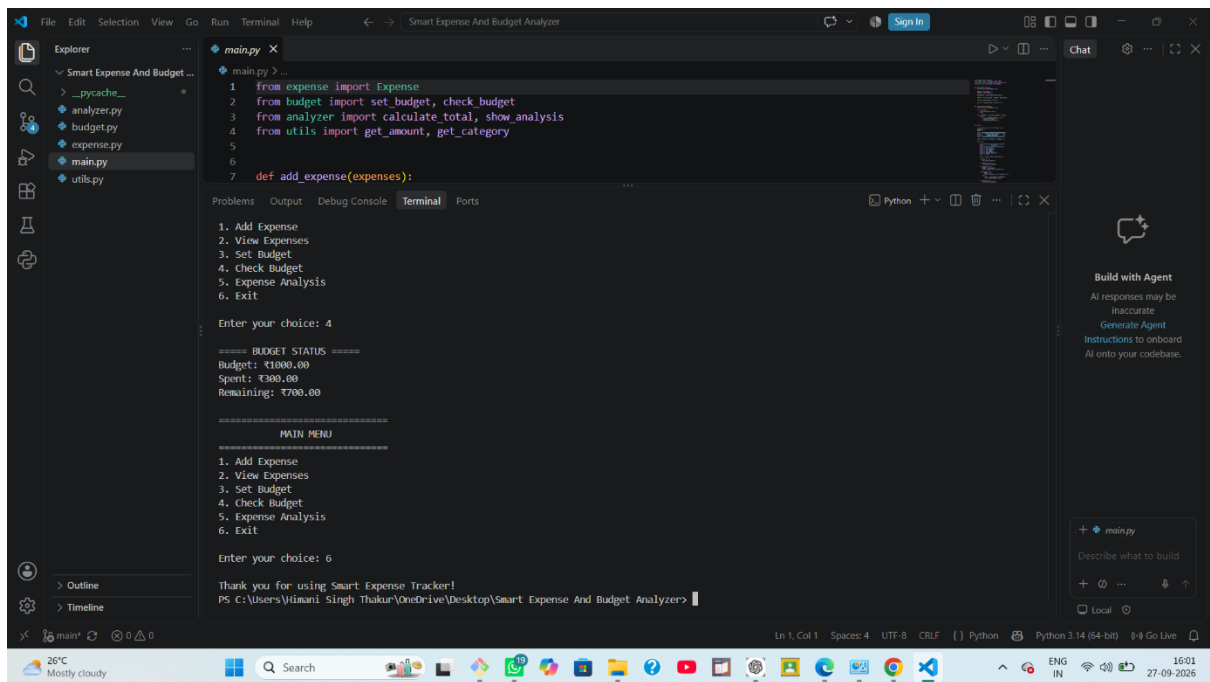
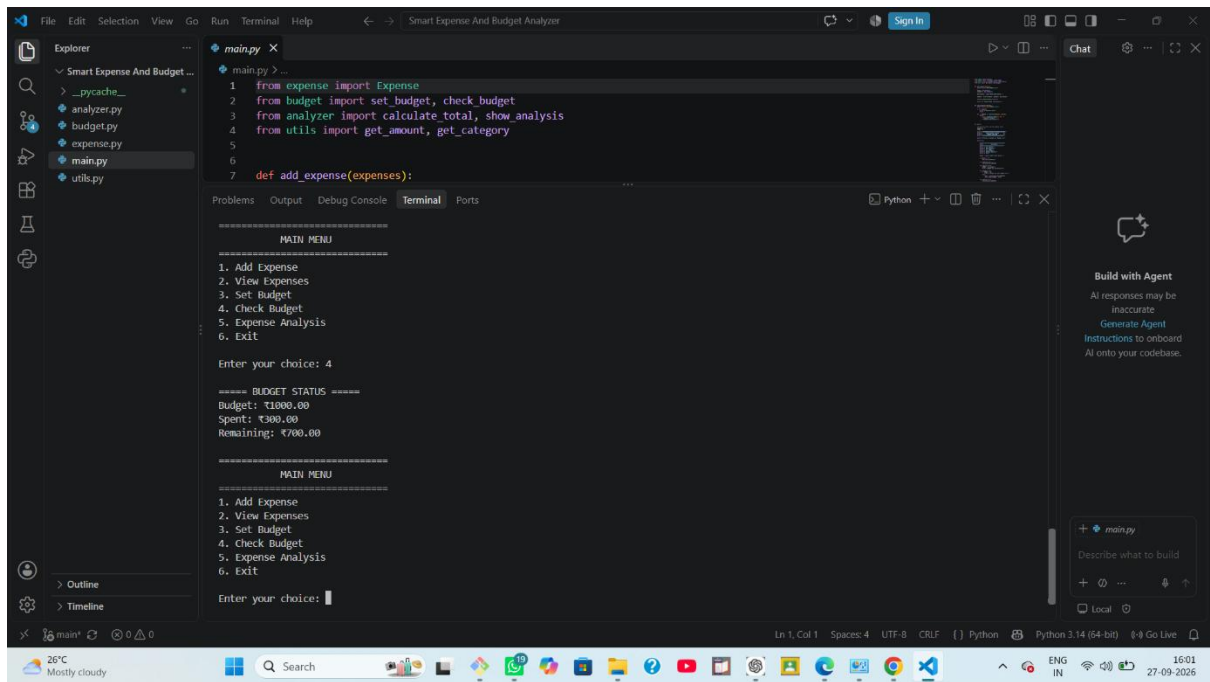
```
=====
MAIN MENU
=====
1. Add Expense
2. View Expenses
3. Set Budget
4. Check Budget
5. Expense Analysis
6. Exit

Enter your choice: 2

===== ALL EXPENSES =====
1. ₹280.00 | Food | Lunch
2. ₹100.00 | Transport | Bus

=====
MAIN MENU
=====
1. Add Expense
2. View Expenses
3. Set Budget
4. Check Budget
5. Expense Analysis
6. Exit

Enter your choice: |
```



10. Testing Approach

The application was tested manually by running it end-to-end and exercising each menu option with both valid and invalid inputs. Key test cases are summarised below:

#	Test Case	Input	Expected Result	Status
1	Add a valid expense	Amount: 500, Category: Food, Desc: Lunch	Expense stored; success message shown	Pass
2	Enter a negative/zero amount	Amount: -50 or 0	"Amount must be greater than zero." re-prompt	Pass
3	Enter non-numeric amount	Amount: abc	"Please enter a valid amount." re-prompt	Pass
4	Choose invalid category number	Category choice: 9	"Please choose a valid option." re-prompt	Pass
5	Check budget before setting one	Menu choice: 4 (budget = 0)	"Please set your budget first."	Pass
6	Check budget after overspending	Budget: 1000, Spent: 1500	"Over budget by: ₹500.00"	Pass
7	View expenses with none added	Menu choice: 2 (empty list)	"No expenses found."	Pass
8	Run analysis with no expenses	Menu choice: 5 (empty list)	"No expenses available for analysis."	Pass
9	Select an undefined menu option	Choice: 9	"Invalid choice. Please try again."	Pass

11. Challenges Faced

- Designing input validation that re-prompts the user cleanly on bad input, instead of crashing the program, required careful use of while True loops combined with try/except blocks.
- Avoiding a divide-by-zero error in the percentage calculation inside show_analysis() when there were no recorded expenses, which is guarded against with an explicit empty-list check before analysis runs.
- Keeping the category list consistent between input collection (utils.py) and analysis grouping (analyzer.py), since a mismatch would silently create a new, unintended category.
- Deciding the right level of separation between modules — balancing having enough files to demonstrate modular design against over-fragmenting a relatively small application.
- Formatting currency values and Unicode symbols (₹) consistently across all print statements and f-strings.

12. Learnings & Key Takeaways

- Gained practical experience structuring a Python program into cohesive, single-responsibility modules rather than one large script.
- Strengthened understanding of object-oriented programming by modelling a real-world entity (an expense) as a class with its own data and behaviour.
- Learned to design defensive input-handling logic that anticipates and gracefully recovers from invalid user input.
- Practised translating a working codebase into formal design artefacts — architecture, use case, sequence, and class diagrams — which clarified the system's structure even to its own author.
- Understood the trade-offs of an in-memory data model versus persistent storage, and when each is appropriate.

13. Future Enhancements

- Add persistent storage using a file (CSV/JSON) or a database (SQLite) so data is saved between sessions, which would also enable an ER diagram for the storage design.
- Build a graphical interface using Tkinter, or a web interface using Flask/Django.
- Add data visualization (pie/bar charts) using a library such as Matplotlib.
- Support multiple users with basic login and per-user budgets.
- Add monthly/yearly summary reports, exportable to PDF or Excel.
- Send a budget-alert notification when spending crosses a configurable threshold.
- Add automated unit tests (e.g., with pytest) for the validation and analysis functions.

14. References

- Python Official Documentation — <https://docs.python.org/3/>
- GeeksforGeeks — Python Programming Tutorials — <https://www.geeksforgeeks.org/python-programming-language/>
- W3Schools — Python Tutorial — <https://www.w3schools.com/python/>

- Course lecture material and reference textbooks as prescribed by the subject curriculum.